MITRC
MODERN INSTITUTE OF TECHNOLOGY AND RESEARCH CENTRE

Name of Faculty: Chakravarty Sharma

Subject & Subject Code: Introduction to Python Programming & 4CS4-05

Semester: IV **Session:** 2022-23

Unit No.: 4 **Date of Submission:** 31/03/2023

Lecture No.	Topics Covered	Page No.	Total Page
25	Functions: Types, parameters, positional arguments, keyword arguments, parameter with default values.	2- 6	5
26	Functions with arbitrary arguments, scope of variables: local and global scope,	7-9	3
27	Functions related programs with arguments & return type	10-13	4
28	Recursion and lambda functions.	14-17	4
29	Function related programs	18-23	6
30	Files: Open, read, write, append and close. Tell and seek methods	24-27	4
31	File related programs – Creation & reading of file	28-34	7
32	Use of Seek(): Move File Pointer Position & related methods	35-42	8

References:

1. Programming Python ny Mark Lutz, O`Reilly

2. Intrduction to Programming using Python by Y.Daniel Liang, Pearson, 2012

01/04/2023

Faculty **HOD** **Dean Academics**

Lecture No. 25

Functions: Types, parameters, positional arguments, keyword arguments, parameter with default values.

Python: user defined functions

A function is a block of program statements which can be used repetitively in a program. It saves the time of a developer. In Python concept of function is same as in other languages. There are some built-in functions which are part of Python. Besides that, we can define functions according to our need.

User defined function

- In Python, a user-defined function's declaration begins with the keyword def and followed by the function name.
- The function may take arguments(s) as input within the opening and closing parentheses, just after the function name followed by a colon.
- After defining the function name and arguments(s) a block of program statement(s) start at the next line and these statement(s) must be indented.

Here is the syntax of a user defined function.

Syntax:

```
def function_name(argument1, argument2, ...) :
    statement_1
    statement_2
    ....
```

Call a function

Calling a function in Python is similar to other programming languages, using the function name, parenthesis (opening and closing) and parameter(s). See the syntax, followed by an example.

Syntax:

```
function_name(arg1, arg2)
```

Example:

```
def avg_number(x, y):
    print("Average of ",x," and ",y, " is ",(x+y)/2)
avg_number(3, 4)
```

Output:

Average of 3 and 4 is 3.5

Explanation:

1. Lines 1-2 : Details (definition) of the function.
2. Line 3 : Call the function.
3. Line 1 : Pass parameters : x = 3, y = 4
4. Line 2 : Print the value of two parameters as well as their average value.

Function without arguments:

The following function has no arguments.

```
def function_name() :
    statement_1
    statement_2
    ....
```

Example:

```
def printt():
    print("This is Python 3.2 Tutorial")
    print("This is Python 3.2 Tutorial")
    print("This is Python 3.2 Tutorial")
printt()
```

Output:

This is Python 3.2 Tutorial

This is Python 3.2 Tutorial

This is Python 3.2 Tutorial

Explanation:

1. Lines 1-4 : Details (definition) of the function.
2. Line 5 : Call the function.
3. Line 1 : No parameter passes.
4. Line 2-4 : Execute three print statements.

The Return statement in function

In Python the return statement (the word return followed by an expression.) is used to return a value from a function, return statement without an expression argument returns none. See the

Syntax.

```
def function_name(argument1, argument2, ...) :
   statement_1
   statement_2
   ....
   return expression
```

Calling to function

```
function_name(arg1, arg2)
```

Example:

The following function returns the square of the sum of two numbers.

```
def nsquare(x, y):
   return (x*x + 2*x*y + y*y)

print("The square of the sum of 2 and 3 is : ", nsquare(2, 3))
```

Output:

The square of the sum of 2 and 3 is : 25

Explanation:

1. Lines 1-2 : Details (definition) of the function.
2. Line 3 : Call the function within a print statement.
3. Line 1 : Pass the parameters x = 2, y = 3
4. Line 2 : Calculate and return the value of $(x + y)^2$

Default Argument Values

In function's parameters list we can specify a default value(s) for one or more arguments. A default value can be written in the format "argument1 = value", therefore we will have the option to declare or not declare a value for those arguments.

Example:

The following function returns the square of the sum of two numbers, where default value of the second argument is 2.

```
def nsquare(x, y = 2):
    return (x*x + 2*x*y + y*y)

print("The square of the sum of 2 and 2 is : ", nsquare(2))
print("The square of the sum of 2 and 3 is : ", nsquare(2,4))
```

Output:

The square of the sum of 2 and 2 is: 16

The square of the sum of 2 and 4 is : 36

Explanation:

Lines 1-2 : Details (definition) of the function.

For first print statement [Line no 3]

Line 3 : Call the function without a second argument, within a print statement.

Line 1 : Pass the parameters x = 2, default value.

Line 2 : Calculate and return the value of $(x + y)^2$

For second print statement [Line no 4]

Line 3 : Call the function with all arguments, within a print statement.

Line 1 : Pass the parameters x = 2, y = 4.

Line 2 : Calculate and return the value of $(x + y)^2$

Keyword Arguments:

Functions can also be called using **keyword arguments**. Arguments which are preceded with a variable name followed by a '=' sign (e.g. var_name='') are called **keyword arguments**.

All the keyword arguments passed must match one of the arguments accepted by the function. You may change the order of appearance of the keyword.

Example:

```
def marks(english, math = 85, science = 80):
        print('Marks in : English is - ', english,', Math - ',math, ', Science - ',science)
marks(71, 77)
marks(65, science = 74)
marks(science = 70, math = 90, english = 75)
```

Output:

```
Marks in : English is -  71 , Math -  77 , Science -  80
Marks in : English is -  65 , Math -  85 , Science -  74
Marks in : English is -  75 , Math -  90 , Science -  70
```

Lecture No. 26

Functions with arbitrary arguments, scope of variables: local and global scope,

Arbitrary Arguments, *args

If you do not know how many arguments that will be passed into your function, add a * before the parameter name in the function definition.

This way the function will receive a *tuple* of arguments, and can access the items accordingly:

Example

If the number of arguments is unknown, add a * before the parameter name:

```
def my_function(*kids):
  print("The youngest child is " + kids[2])


my_function("Raj", "Mukesh", "Suraj")
```

Output

The youngest child is Suraj

Python Scope

A variable is only available from inside the region it is created. This is called scope.

Local Scope

A variable created inside a function belongs to the local scope of that function, and can only be used inside that function.

Example

A variable created inside a function is available inside that function:

```
def myfunc():
  x = 300
  print(x)
myfunc()
```

Function Inside Function

As explained in the example above, the variable x is not available outside the function, but it is available for any function inside the function:

Example

The local variable can be accessed from a function within the function:

```
def myfunc():
  x = 300
  def myinnerfunc():
    print(x)
  myinnerfunc()
myfunc()
```

Global Scope

A variable created in the main body of the Python code is a global variable and belongs to the global scope.

Global variables are available from within any scope, global and local.

Example

A variable created outside of a function is global and can be used by anyone:

```
x = 300 # Global Scope
def myfunc():
  print(x)
myfunc()
print(x)
```

Naming Variables

If you operate with the same variable name inside and outside of a function, Python will treat them as two separate variables, one available in the global scope (outside the function) and one available in the local scope (inside the function):

Example

The function will print the local x, and then the code will print the global x:

```
x = 300
```

```
def myfunc():
  x = 200
  print(x)
myfunc()
print(x)
```

Global Keyword

If you need to create a global variable, but are stuck in the local scope, you can use the global keyword.

The global keyword makes the variable global.

Example

If you use the global keyword, the variable belongs to the global scope:

```
def myfunc():
  global x
  x = 300
myfunc()
print(x)
```

Also, use the global keyword if you want to make a change to a global variable inside a function.

Example

To change the value of a global variable inside a function, refer to the variable by using the global keyword:

```
x = 300
def myfunc():
  global x
  x = 200
myfunc()
print(x)bitrary Arguments are often shortened to *args in Python documentations.
```

Lecture No. 27

Functions related programs

```
# defining a function to calculate LCM
    def calculate_lcm(x, y):
        # selecting the greater number
        if x > y:
            greater = x
        else:
            greater = y
        while(True):
            if((greater % x == 0) and (greater % y == 0)):
                lcm = greater
                break
            greater += 1
        return lcm

    # taking input from users
    num1 = int(input("Enter first number: "))
    num2 = int(input("Enter second number: "))
    # printing the result for the users
    print("The L.C.M. of", num1,"and", num2,"is", calculate_lcm(num1, num2))
```

Output:

```
Enter first number: 3
Enter second number: 4
The L.C.M. of 3 and 4 is 12
```

Example:

```
Select operation -
```

a. Add

b. Subtract

c. Multiply

d. Divide

Select operations form a, b, c, d: "c"

Please enter first number: 11

Please enter second number: 4

11 * 4 = 44

Code: for Simple Calculator

```
def add(P, Q):
  # This function is used for adding two numbers
  return P + Q
def subtract(P, Q):
  # This function is used for subtracting two numbers
  return P - Q
def multiply(P, Q):
  # This function is used for multiplying two numbers
  return P * Q
def divide(P, Q):
  # This function is used for dividing two numbers
  return P / Q
# Now we will take inputs from the user
print ("Please select the operation.")
print ("a. Add")
print ("b. Subtract")
print ("c. Multiply")
print ("d. Divide")

choice = input("Please enter choice (a/ b/ c/ d): ")
```

```
num_1 = int (input ("Please enter the first number: "))
num_2 = int (input ("Please enter the second number: "))

if choice == 'a':
   print (num_1, " + ", num_2, " = ", add(num_1, num_2))

elif choice == 'b':
   print (num_1, " - ", num_2, " = ", subtract(num_1, num_2))

elif choice == 'c':
   print (num1, " * ", num2, " = ", multiply(num1, num2))
elif choice == 'd':
   print (num_1, " / ", num_2, " = ", divide(num_1, num_2))
else:
   print ("This is an invalid input")
```

Output:

Case - (1):

Please select the operation.

a. Add

b. Subtract

c. Multiply

d. Divide

Please enter choice (a/ b/ c/ d): d

Please enter the first number: 1

Please enter the second number: 2

1 / 2 = 0.5

Case - (2):

Please select the operation.

a. Add

b. Subtract

c. Multiply

d. Divide

Please enter choice (a/ b/ c/ d): b

Please enter the first number: 12

Please enter the second number: 11

12 - 11 = 1

Python Program to Display Fibonacci Sequence Using Recursion

Fibonacci sequence:

A Fibonacci sequence is a sequence of integers which first two terms are 0 and 1 and all other terms of the sequence are obtained by adding their preceding two numbers.

For example: 0, 1, 1, 2, 3, 5, 8, 13 and so on...

```
def recur_fibo(n):
  if n <= 1:
     return n
  else:
     return(recur_fibo(n-1) + recur_fibo(n-2))
# take input from the user
nterms = int(input("How many terms? "))
# check if the number of terms is valid
if nterms <= 0:
  print("Plese enter a positive integer")
else:
  print("Fibonacci sequence:")
  for i in range(nterms):
     print(recur_fibo(i))
```

Lecture No. 28

Recursion And Lambda Function.

Lambda Function

In Python, small anonymous (unnamed) functions can be created with lambda keyword. Lambda forms can be used as an argument to other function where function objects are required but syntactically they are restricted to a single expression

A lambda function can take any number of arguments, but can only have one expression.

Syntax

lambda arguments : expression

The expression is executed and the result is returned:

Example

Add 10 to argument a, and return the result:

```
x = lambda a : a + 10
print(x(5))
```

Lambda functions can take any number of arguments:

Example

Multiply argument a with argument b and return the result:

```
x = lambda a, b : a * b
print(x(5, 6))
```

Example

Summarize argument a, b, and c and return the result:

```
x = lambda a, b, c : a + b + c
print(x(5, 6, 2))
```

Why Use Lambda Functions?

The power of lambda is better shown when you use them as an anonymous function inside another function.

Say you have a function definition that takes one argument, and that argument will be multiplied with an unknown number:

```
def myfunc(n):
  return lambda a : a * n
```

Use that function definition to make a function that always doubles the number you send in:

Example

```
def myfunc(n):
  return lambda a : a * n
  mydoubler = myfunc(2)
print(mydoubler(11))
```

Or, use the same function definition to make a function that always triples the number you send in:

Example

```
def myfunc(n):
  return lambda a : a * n
mytripler = myfunc(3)
print(mytripler(11))
```

Or, use the same function definition to make both functions, in the same program:

Example

```
def myfunc(n):
  return lambda a : a * n

mydoubler = myfunc(2)
mytripler = myfunc(3)
```

```
print(mydoubler(11))
print(mytripler(11))
```

Use lambda functions when an anonymous function is required for a short period of time..

A function like this:

```
def average(x, y):
   return (x + y)/2
print(average(4, 3))
```

may also be defined using lambda

```
print((lambda x, y: (x + y)/2)(4, 3))
```

Output:

3.5

Count Down to Zero

The first example is a function called countdown(), which takes a positive number as an argument and prints the numbers from the specified argument down to zero:

```
def countdown(n):
     print(n)
     if n == 0:
        return           # Terminate recursion
    else:
     countdown(n - 1)   # Recursive call
countdown(5)
```

Output

5
4
3
2
1
0

Notice how countdown() fits the paradigm for a recursive algorithm described above:

- The base case occurs when n is zero, at which point recursion stops.
- In the recursive call, the argument is one less than the current value of n, so each recursion moves closer to the base case.

Example

Calculate factorial with function recursion

```
def factorial(n):
    if n <= 1:
        return 1
    else:
        n * factorial(n - 1)

factorial(4)
```

Output

24

Lecture No. 29

Practice of function related programs

Python Program to check Armstrong Number

Suppose we have a given number, "K", and we have to check whether the given number is an Armstrong number or not. In this tutorial, we will learn how to identify the given number is an Armstrong number or not using Python.

Definition:

A positive number of "p" number of digits is called an Armstrong number of order n (order is the number of digits) if:

1. abcde? = pow(a, p) + pow(b, p) + pow(c, p) + pow(d, p) + pow(e, p) + ?

Example:

1. Input: 407
2. Output: Yes, 407 **is** an Armstrong number
3. Solution:
4. (4 * 4 * 4) + (0 * 0 * 0) + (7 * 7 * 7) = 407
5. Input: 1634
6. Output: Yes, 1634 **is** an Armstrong number
7. Solution:
8. (1 * 1 * 1) + (6 * 6 * 6) + (3 * 3 * 3) + (4* 4 * 4) = 1634
9. Input: 253
10. Output: No, 253 **is not** an Armstrong number
11. Solution:
12. (2 * 2 * 2) + (5 * 5 * 5) + (3 * 3 * 3) = 160
13. Input: 1
14. Output: Yes, 1 **is** an Armstrong number
15. Solution:
16. (1* 1) = 1

Example:

```
def power_1(A, B):

    if B == 0:
        return 1
    if B % 2 == 0:
        return power_1(A, B // 2) * power(A, B // 2)

    return A * power(A, B // 2) * power(A, B // 2)

# Function for calculating "order of the number"
def order_1(A):

    # Variable for storing the number
    N = 0
    while (A != 0):
        N = N + 1
        A = A // 10

    return N

# Function for checking if the given number is Armstrong number or not
def is_Armstrong(A):

    N = order_1(A)
    temp_1 = A
    sum_1 = 0

    while (temp_1 != 0):
```

```
    R_1 = temp_1 % 10
    sum_1 = sum_1 + power_1(R_1, N)
    temp_1 = temp_1 // 10

  # If the above condition is satisfied, it will return the result
  return (sum_1 == A)

# Driver code
A = int(input("Please enter the number to be checked: "))
print(is_Armstrong(A))
```

Output:

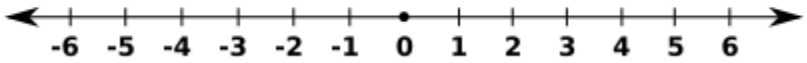
```
1#
Please enter the number to be checked:  0
True
2#
Please enter the number to be checked:  123
False
3#
Please enter the number to be checked:  1634
True
```

Python Program to check if a Number is Positive, Negative or Zero

Here, we will learn to write a Python program through which we can check if the given number is positive, zero, or negative.

Positive Numbers: A number is said to be positive if the number has a value greater than 0, like 1, 2, 3, 5, 7, 9, 11, 13, etc. All the natural numbers are positive numbers.

Negative Numbers: If a given number has a value less than 0 like -1, -2, -3, -5, -7, -9, -11, -13, etc., then we can say that the given number is a negative number. Only rational and integer type numbers can have negative values or numbers.



Let us look at the following example to understand the implementation

Example:

```
# Default function to run if else condition
def NumberCheck(a):
    # Checking if the number is positive
    if a > 0:
        print("Number given by you is Positive")
    # Checking if the number is negative
    elif a < 0:
        print("Number given by you is Negative")
    # Else the number is zero
    else:
        print("Number given by you is zero")
# Taking number from user
a = float(input("Enter a number as input value: "))
# Printing result
NumberCheck(a)
```

Output:

```
Enter a number as input value: -6
Number given by you is Negative
```

Python program to convert Kilometres to Miles

In this tutorial, we will learn a step-by-step approach for how to write a Python program to convert kilometres value into miles value.

Understanding the Logic Behind the Conversion of Kilometre to Miles

Let's begin with the basics; first, we will discuss the measuring units. As we know, Kilometre and Miles represent the unit of length.

1 kilometre equals 0.62137 miles.

Miles = kilometre * 0.62137

And,

Kilometre = Miles / 0.62137

Hence, the value "0.62137" would be considered as the factor of conversation or the ratio for converting units.

Steps to be Followed for Converting Kilometre unit to Miles Unit.

We follow the below steps for getting a clear idea of how to convert kilometres to miles units.

Step 1: We will define the variable for storing the value of kilometre_1 and accepting the input from the user.

```
1. kilometre_1 = float (input ("Please enter the speed of car in Kilometre as a unit: "))
```

Step 2: Now, we will define and store the conversion factor into the variable.

```
1. conversion_ratio_1 = 0.621371
```

Step 3: Then, we will define the variable for storing the value of kilometre_1 which is converted into miles. Then in addition we will write the logic of converting kilometre to miles units.

```
1. miles_1 = kilometre_1 * conversion_ratio_1
```

Step 4: At last, we will display the converted value by using the print() function.

```
1. print ("The speed value of car in Miles: ", miles_1)
```

Full Extension of Code:

```
1. kilometre_1 = float (input ("Please enter the speed of car in Kilometre as a unit: "))
2. conversion_ratio_1 = 0.621371
3. miles_1 = kilometre_1 * conversion_ratio_1
```

4. **print** ("The speed value of car in Miles: ", miles_1)

Output:

Please enter the speed of car in Kilometre as a unit: 16

The speed value of car in Miles: 9.941936

Approach 2:

We can also use another approach of simply statistically defining a function for converting kilometres to miles unit.

Example:

```
1. def kilometre_1(km):
2.     conversion_ratio_1= 0.621371
3.     miles_1 = km * conversion_ratio_1
4.     print ("The speed value of car in Miles: ", miles_1)
5. km = float (input ("Please enter the speed of car in Kilometre as a unit: "))
6. kilometre_1(km)
```

Output:

Please enter the speed of car in Kilometre as a unit: 14

The speed value of car in Miles: 8.699194

Lecture No. 30

Files: Open, read, write, append and close. Tell and seek methods

Python File Open

File handling is an important part of any web application.

Python has several functions for creating, reading, updating, and deleting file

File Handling

The key function for working with files in Python is the open() function.

The open() function takes two parameters; *filename*, and *mode*.

There are four different methods (modes) for opening a file:

"r" - Read - Default value. Opens a file for reading, error if the file does not exist

"a" - Append - Opens a file for appending, creates the file if it does not exist

"w" - Write - Opens a file for writing, creates the file if it does not exist

"x" - Create - Creates the specified file, returns an error if the file exists

In addition you can specify if the file should be handled as binary or text mode

"t" - Text - Default value. Text mode

"b" - Binary - Binary mode (e.g. images)

Syntax

To open a file for reading it is enough to specify the name of the file:

```
f = open("demofile.txt")
```

The code above is the same as:

```
f = open("demofile.txt", "rt")
```

Because "r" for read, and "t" for text are the default values, you do not need to specify them.

Note: Make sure the file exists, or else you will get an error.

Python File Open

Assume we have the following file, located in the same folder as Python:

demofile.txt

Hello! Welcome to demofile.txt

This file is for testing purposes.

Good Luck!

To open the file, use the built-in open() function.

The open() function returns a file object, which has a read() method for reading the content of the file:

Example

```
f = open("demofile.txt", "r")
print(f.read())
```

If the file is located in a different location, you will have to specify the file path, like this:

Example

Open a file on a different location:

```
f = open("D:\\myfiles\welcome.txt", "r")
print(f.read())
```

Read Only Parts of the File

By default the read() method returns the whole text, but you can also specify how many characters you want to return:

Example

Return the 5 first characters of the file:

```
f = open("demofile.txt", "r")
```

Write to an Existing File

To write to an existing file, you must add a parameter to the open() function:

"a" - Append - will append to the end of the file

"w" - Write - will overwrite any existing content

Example

Open the file "demofile2.txt" and append content to the file:

```
f = open("demofile2.txt", "a")
f.write("Now the file has more content!")
f.close()

#open and read the file after the appending:
f = open("demofile2.txt", "r")
print(f.read())
```

Example

Open the file "demofile3.txt" and overwrite the content:

```
f = open("demofile3.txt", "w")
f.write("Woops! I have deleted the content!")
f.close()

#open and read the file after the overwriting:
f = open("demofile3.txt", "r")
print(f.read())
```

Note: the "w" method will overwrite the entire file.

Create a New File

To create a new file in Python, use the open() method, with one of the following parameters:

"x" - Create - will create a file, returns an error if the file exist

"a" - Append - will create a file if the specified file does not exist

"w" - Write - will create a file if the specified file does not exist

Example

Create a file called "myfile.txt":

```
f = open("myfile.txt", "x")
```

Result: a new empty file is created!

Example

Create a new file if it does not exist:

```
f = open("myfile.txt", "w")
```

Delete a File

To delete a file, you must import the OS module, and run its os.remove() function:

Example

Remove the file "demofile.txt":

```
import os
os.remove("demofile.txt")
```

Check if File exist:

To avoid getting an error, you might want to check if the file exists before you try to delete it:

Example

Check if file exists, *then* delete it:

```
import os
if os.path.exists("demofile.txt"):
  os.remove("demofile.txt")
else:
  print("The file does not exist")
```

Delete Folder

To delete an entire folder, use the os.rmdir() method:

Example

Remove the folder "myfolder":

```
import os
os.rmdir("myfolder")
```

Note: You can only remove *empty* folders.

Lecture No. 31

Practice of file related programs

Create File in Python

Python is widely used in data analytics and comes with some inbuilt functions to work with files. We can create a file and do different operations, such as write a file and read a file using Python.

After reading this tutorial, you'll learn: –

- Create a file in the current directory or a specified directory
- Create a file if not exists
- Create a file with a date and time as its name
- Create a file with permissions

Table of contents

- Create A Empty Text File
- Create File In A Specific Directory
- Create a File If Not Exists
- Create File with a DateTime
- Create a file with Permission

Create A Empty Text File

We don't have to import any module to create a new file. We can create a file using the built-in function open().

open('file_Path', 'access_mode')

Pass the file name and access mode to the open() function to create a file. Access mode specifies the purpose of opening a file.

Below is the list of access modes for creating an a file.

File Mode	Meaning
w	Create a new file for writing. If a file already exists, it truncates the file first. Use to create and write content into a new file.

File Mode	Meaning
x	Open a file only for exclusive creation. If the file already exists, this operation fails.
a	Open a file in the append mode and add new content at the end of the file.
b	Create a binary file
t	Create and open a file in a text mode

File access mode

Example: **Create a new empty text file** named 'sales.txt'

```
# create a empty text file
# in current directory
fp = open('sales.txt', 'x')
fp.close()
```

Use **access mode w** if you want to create and write content into a file.

```
# create a empty text file
fp = open('sales_2.txt', 'w')
fp.write('first line')
fp.close()
```

As you can see in the image two new files gets created in the account folder.

created files

Note:

- The file is created in the same directory where our program/script is running.
- If you have not specified any specific path(directory location), the file is created in the working directory. It is known as creating a file using the **relative path**. A relative path contains the current directory and then the file name.

You can **verify the result** using the following four approaches

1. If the script executed without an error or exception
2. By checking the working directory manually to look for a new file
3. Use the os.listdir(directory_path) function to list all files from a folder before and after creating a file

4. Use the os.path.isfile(file_path) function to verify if a newly created file exists in a directory.

Let's verify our operation result.

```
import os

# list files from a working directory
print(os.listdir())

# verify file exist
print(os.path.isfile('sales.txt'))
```

Output

```
['sample.txt', 'sales.txt', 'sales_2.txt']
True
```

Create File In A Specific Directory

To create a file inside a specific directory, we need to open a file using the absolute path. An absolute path contains the entire path to the file or directory that we need to use.

It includes the **complete directory list required to locate the file**. For example, /user/Pynative/data/sales.txt is an absolute path to discover the sales.txt. All of the information needed to find the file is contained in the path string.

Let's see the example to **create a file for writing** using the absolute path.

```
# create a text file for writing
with open(r'E:\pynative\reports\profit.txt', 'w') as fp:
    fp.write('This is first line')
    pass
```

Note: Using the with statement a **file is closed automatically** it ensures that all the resources that are tied up with the file are released.

Let's verify result using the absolute path.

```
import os
```

```
# list files a directory
print(os.listdir(r'E:\pynative\reports'))
# output ['sample.txt', 'sales.txt', 'sales_2.txt' 'profit.txt']

# verify file exist
print(os.path.isfile(r'E:\pynative\reports\profit.txt'))
# output True
```

Also, you can **join directory path and file name** to create file at the specified location.

If you have a directory path and file name in two variables, use the os.path.join() function to construct a full path. This function accepts the directory path and file name as arguments and constructs an absolute path to create a file.

Example:

```
import os

# Specify the directory path
path = r'E:\pynative\account'
file_name = 'revenue.txt'

# Creating a file at specified folder
# join directory and file path
with open(os.path.join(path, file_name), 'w') as fp:
    # uncomment below line if you want to create an empty file
    fp.write('This is a new line')
```

Create a File If Not Exists

Sometimes it is essential not to create a new file if a file with the same name already exists in a given path. By default, when you open a file in write mode, it overwrites it if it exists. Else, create the new one.

We can create a file only if it is not present using the following two ways:

- Use os.path.exists("file_path") function to check if a file exists.

- Use the access mode x in the open() function and exception handling.

Example 1: create file if not exists.

```
import os


file_path = r'E:\pynative\account\profit.txt'
if os.path.exists(file_path):
    print('file already exists')
else:
    # create a file
    with open(file_path, 'w') as fp:
        # uncomment if you want empty file
        fp.write('This is first line')
```

Example 2: Use file access mode x

The access mode x open a file for exclusive creation. If the file already exists, this operation fails with FileExistsError. Use try-except block to handle this error.

```
try:
    file_path = r'E:\pynative\account\profit.txt'
    # create file
    with open(file_path, 'x') as fp:
        pass
except:
    print('File already exists')
```

Create File with a DateTime

Let's see how to create a text file with the current date as its name. Use the datetime module to get the current date and time and assign it to the file name to create a file with the date and time in its name.

- Python provides a datetime module that has several classes to access and manipulate the date and timestamp value.
- First, get the current datetime value

- Next, we need to format datetime into a string to use it as a file name.
- At last, pass it to the open() function to create a file

Example

```
from datetime import datetime

# get current date and time
x = datetime.now()

# create a file with date as a name day-month-year
file_name = x.strftime('%d-%m-%Y.txt')
with open(file_name, 'w') as fp:
    print('created', file_name)

# with name as day-month-year-hours-minutes-seconds
file_name_2 = x.strftime('%d-%m-%Y-%H-%M-%S.txt')
with open(file_name_2, 'w') as fp:
    print('created', file_name_2)

# at specified directory
file_name_3 = r"E:\demos\files_demos\account\\" + x.strftime('%d-%m-%Y-%H-%M-%S.txt')
with open(file_name_3, 'w') as fp:
    print('created', file_name_3)
```

Output:

```
created 29-06-2021.txt
created 29-06-2021-14-57-24.txt
created E:\demos\files_demos\account\\29-06-2021-14-57-24.txt
```

Create a file with Permission

Let's see how to create a file with permissions other users can write.

- To create a file with appropriate permissions, use os.open() to create the file descriptor and set the permission.
- Next, open the descriptor using the built-in function open()

```
import os

file_path = r'E:\pynative\account\sample.txt'
# The default umask is 0o22 which turns off write permission of group and others
os.umask(0)
with open(os.open(file_path, os.O_CREAT | os.O_WRONLY, 0o777), 'w') as fh:
    fh.write('content')
```

Lecture No. 32

Practice of file related programs

Python File Seek(): Move File Pointer Position

Python offers several methods for file handling. In addition to the standard operations like reading and writing to the files, there are methods to manipulate the file pointer effectively.

In this tutorial, you'll learn how to use the seek() function to **move the position of a file pointer** while reading or writing a file.

Table of contents

- What is seek() in Python
- How to Use seek() Method
  - Example
- Seek the Beginning of the File
- Seeking The End of File
- Seek From The Current Position
- Seek backward With Negative Offset
- tell() Function To Get File Handle Position
- Summary

What is seek() in Python

The seek() function **sets the position of a file pointer** and the tell() function **returns the current position** of a file pointer.

A **file handle or pointer denotes the position from which the file contents will be read or written**. File handle is also called as file pointer or cursor.

For example, when you open a file in write mode, the file pointer is placed at the 0th position, i.e., at the start of the file. However, it changes (increments) its position as you started writing content into it.

Or, when you read a file line by line, the file pointer moves one line at a time.

Sometimes we may have to read only a specific portion of the file, in such cases use the seek() method to move the file pointer to that position.

For example, use the seek() function to do the file operations like: –

- Read a file from the 10th character.
- Directly jump to the 5th character from the end of the file.
- Add new content to file after a particular position.

How to Use seek() Method

To change the file handle's position use seek() method. As we discussed, the seek() method sets the file's current position, and then we can read or write to the file from that position.

Syntax:

f.seek(offset, whence)

How many points the pointer will move is **computed from adding offset to a reference point**; the reference point is given by the whence argument.

The allowed values for the whence argument are: –

- A whence value of **0** means from the beginning of the file.
- A whence value of **1** uses the current file position
- A whence value of **2** uses the end of the file as the reference point.

The default value for the whence is the beginning of the file, which is **0**

Refer to the below **table for clear understanding**.

Seek Operation	Meaning
f.seek(0)	Move file pointer to the beginning of a File
f.seek(5)	Move file pointer five characters ahead from the beginning of a file.
f.seek(0, 2)	Move file pointer to the end of a File
f.seek(5, 1)	Move file pointer five characters ahead from the current position.
f.seek(-5, 1)	Move file pointer five characters behind from the current position.
f.seek(-5, 2)	Move file pointer in the reverse direction. Move it to the 5th character from the end

Seek Operation	Meaning
	of the file

File seek function

Example

Consider the following example where we are reading a text file contents with the **offset as 6**. It means we will start **reading the file directly from the 6th character**.

text file

```
with open(r'E:\demos\files_demos\sample.txt', "r") as fp:
    # Moving the file handle to 6th character
    fp.seek(6)
    # read file
    print(fp.read())
```

Output

```
line
Second line
Third line
Fourth line
Fifth line
Sixth line
Seventh line
Eighth line
```

As you can see in the output, the first six characters are missing.

Seek the Beginning of the File

We can move the file pointer to the beginning of the file using the seek() method by passing the setting whence to **0**.

The 0 indicates the first byte, which is the beginning of the file.

Example

Let's see how to bring the file cursor to the beginning of the file.

In this example, we are writing to the text file. After adding content, we wanted to move the pointer to the beginning of the file to read the entire file.

```
# open file in write and read mode w+
with open(r'E:\demos\files_demos\test.txt', "w+") as fp:
    # add content
    fp.write('My First Line\n')
    fp.write('My Second Line')
    # move pointer to the beginning
    fp.seek(0)
    # read file
    print(fp.read())
```

Output

```
My First Line
My Second Line
```

Seeking The End of File

Set whence to **2** and the **offset to 0** to move the file pointer to the end of the file.

- In the below example, we will perform the following three operations
- We will move the file pointer at the end of the file and write new content
- Next, we will move the file pointer at the start of the file and write fresh content at the beginning of the file.
- Again, we will move the file pointer to the end of the file and write more content.

Example:

Let's see how to move the file cursor to the end of the file. We will use the existing file for this operation and open a file in read and write mode.

```
# open file for reading and writing  a+
with open(r'E:\demos\files_demos\test.txt', "r+") as fp:
    # Moving the file handle to the end of the file
    fp.seek(0, 2)
```

```
# Inserting new content to the end of the file
fp.write("\nThis content is added to the end of the file")

# moving to the beginning
# again read the whole file
fp.seek(0)
print(fp.read())
```

Output

My First Line

My Second Line

This content is added to the end of the file

Seek From The Current Position

We can move the file pointer few positions ahead from the current position by setting the whence to 1 and offset to the number of the position you want to move.

For example, the current file pointer is at 20th position, and you wanted to jump to the 75th character then set offset to 50 and whence to 1.

We will use the existing file for this operation and open a file in read and write mode.

Note:

If your using seek() function in text files (those opened without a b in the access mode), only seeks relative to the beginning of the file are allowed.

If you try to move the file handle from the current position you'll get an io.UnsupportedOperation: can't do nonzero cur-relative seeks error.

So **open the file in binary mode if you want to move the file pointer ahead or behind** from the current position

Example: Move the file handle 10 points ahead from current position.

Note:

- **Open file in binary mode.** For reading use the rb, for writing use the wb, and for both reading and writing use rb+.
- **Convert byte to string** if you are reading a text file.

```
# Open file for reading in Binary mode
with open(r'E:\demos\files_demos\test.txt', "rb") as fp:
    # Move the file handle to the 5th character
    # from the beginning of the file
    fp.seek(3)
    # read 5 bytes and convert it to string
    print(fp.read(5).decode("utf-8"))

    # Move the fp 10 points ahead from current position
    # read 5 bytes and convert it to string
    fp.seek(10, 1)
    # again read 6 bytes
    print(fp.read(6).decode("utf-8"))
```

Seek backward With Negative Offset

In some cases, we have to read characters from the end of the file. to do that, we need to move the file pointer in a reverse direction.

Here, you'll learn how to **seek file handle backward from current position as well as from the end of file**.

For example, **move to the 10th character from the end of the file**. This can be done by setting the offset corresponding to the end of the file.

Example:

In this example, we are opening the file in the read binary mode (rb) and pass the offset corresponding to the end of the file.

```
# Open file for reading in binary mode
with open(r'E:\demos\files_demos\test.txt', "rb") as fp:
    # Move in reverse direction
    # move to the 40th character from the end of the file
    fp.seek(-40, 2)
    # read 11 bytes and convert it to string
```

```
    print(fp.read(11).decode("utf-8"))
```

Output

content is

Use the below code **to Seek backwards from current position**

```
# Open file for reading in binary mode
with open(r'E:\demos\files_demos\test.txt', "rb") as fp:
    # read first 8 bytes
    print(fp.read(8).decode('utf-8'))
    # Move in reverse direction
    # move to the 5th behind from current position
    fp.seek(-5, 1)
    # read 10 bytes and convert it to string
    print(fp.read(10).decode("utf-8"))
```

Output:

My First

First Line

tell() Function To Get File Handle Position

While the file's access mode implies the type of operation that we intend to perform in the file, it also determines the filehandle position. For example, if the file is opened in reading, the file handle will be in the beginning, and after reading the entire file, it will be in the last character, which is the End of the File.

We can get the file handle current position using the tell() method.

Syntax:

file_object.tell()

There are no arguments for this method. The **return value is the integer** representing the file handle position.

```
# open file for reading and writing  r+
with open(r'E:\demos\files_demos\test.txt', "r+") as fp:
    # Moving the file handle to the end of the file
```

```
fp.seek(0, 2)
# getting the file handle position
print('file handle at:', fp.tell())
# writing new content
fp.write("\nDemonstrating tell")
# getting the file handle position
print('file handle at:', fp.tell())
# move to the beginning
fp.seek(0)
# getting the file handle position
print('file handle at:', fp.tell())
# read entire file
print('***Printing File Content***')
print(fp.read())
print('***Done***')
# getting the file handle position
print('file handle at:', fp.tell())
```

Output

```
file handle at: 75
file handle at: 95
file handle at: 0
***Printing File Content***
My First Line
My Second Line
This content is added to the end of the file
Demonstrating tell
***Done***
file handle at: 95
```